Manuel Vásquez 3/29/2026

WebDev Assigment 4

Navegación SPA y Estrategia de Estado

Para esta tarea he utilizado de ejemplo mi implementación para el último laboratorio

GitLab Link: https://gitlab.com/SerJunken/webdev_manuelvasquez/-/commits/LB-Tarea4-SharedState-MultiSPA

Es un ejemplo en donde utilizo la API de openLibrary para obtener libros y datos de ellos, buscar libros y tenerlos en una lista

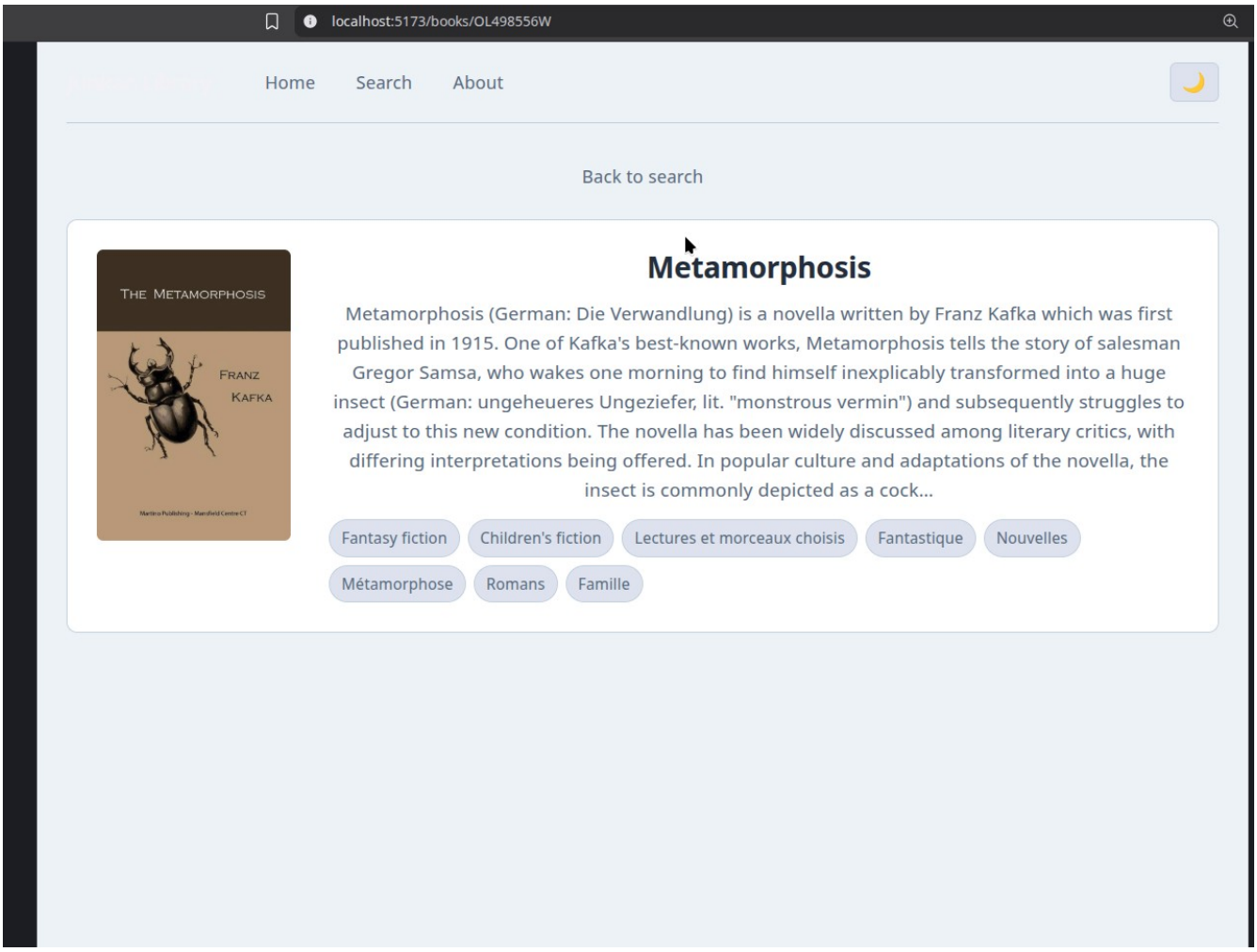
Parte 1: Análisis de React Router: Analiza una configuración de enrutamiento proporcionada para explicar el enrutamiento anidado, rutas dinámicas y coincidencia de rutas.

La configuración de enrutamiento que armé usa rutas anidadas. La idea es que AppLayout es la ruta "padre" y todas las páginas viven dentro de ella. Esto funciona porque React Router renderiza el componente padre primero, y donde yo ponga <Outlet /> es donde aparece el contenido de la ruta hija activa. Así el header y el nav se renderizan una sola vez y nunca desaparecen al cambiar de página.

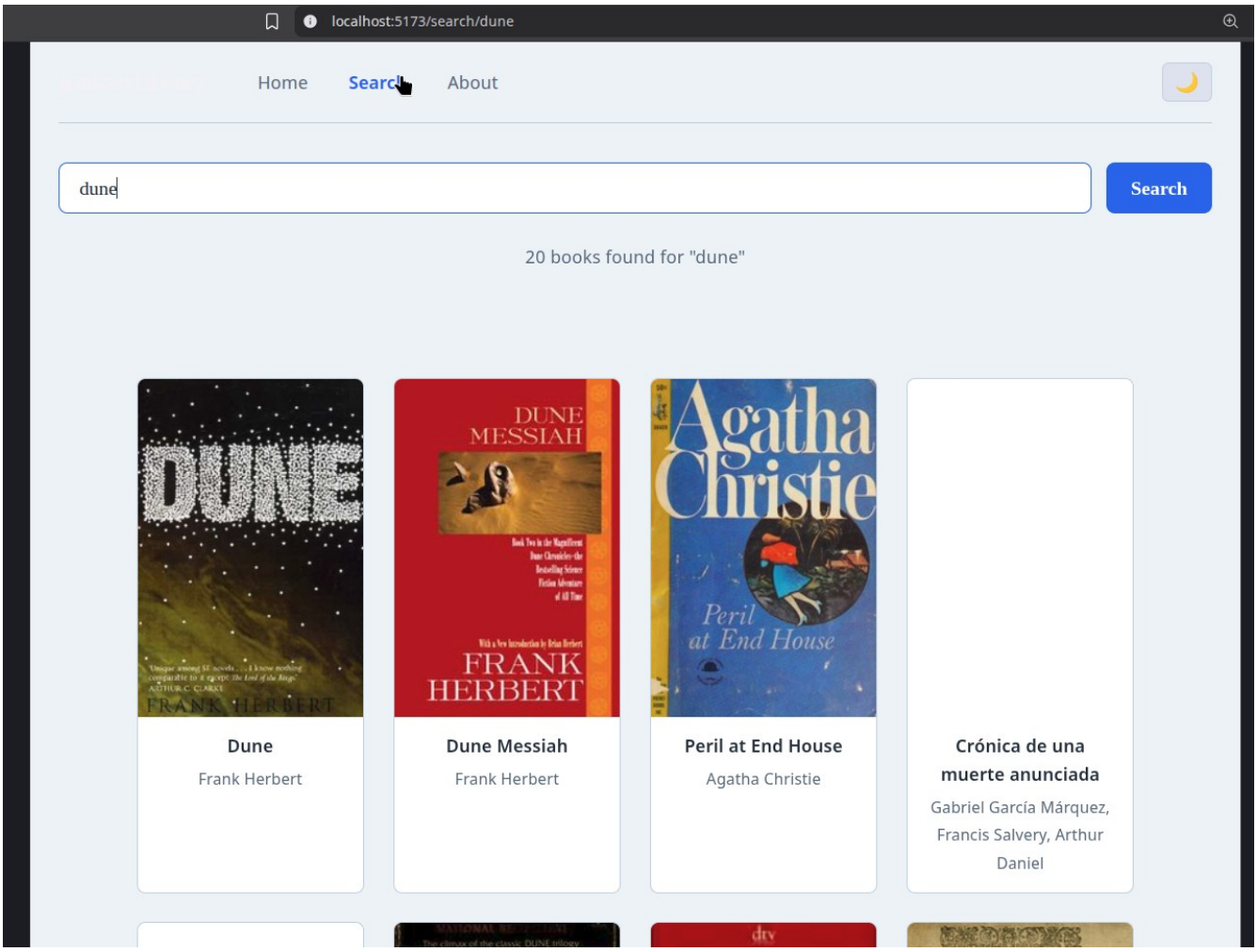
```
export default function App() {
  return (
    <ThemeProvider
      <Suspense fallback={<LoadingFallback />}>
        <Routes>
          <Route element={<AppLayout />}>
            <Route path="/" element={<HomePage />} />
            <Route path="/search" element={<SearchPage />} />
            <Route path="/search/:initialSearchTerm" element={<SearchPage />} />
            <Route path="/books/:bookId" element={<BookDetailPage />} />
            <Route path="/about" element={<AboutPage />} />
            <Route path="*" element={
              <div style={{ textAlign: 'center', padding: '3rem', color: 'var(--text-muted)' }}
                <h2>404 Page not found</h2>
              </div>
            } />
          </Route>
        </Routes>
      </Suspense>
    </ThemeProvider>
  )
}
```

Las rutas que definí son /, /search, /search/:initialSearchTerm, /books/:bookId y /about. La ruta /books/:bookId es dinámica: el segmento :bookId es un parámetro que cambia según el libro que el usuario clickeó. Adentro del componente lo leo con useParams() y lo uso directamente para hacer el fetch. Por ejemplo, si el usuario entra a /books/OL45804W, React Router extrae OL45804W y yo lo paso a la URL de la API de Open Library.

Link: http://localhost:5173/books/OL498556W



La ruta /search/:initialSearchTerm es un caso interesante porque es básicamente la misma página que /search, solo que con el parámetro pre-cargado. Decidí hacer dos rutas separadas en vez de hacer el parámetro opcional porque así es más explícito y evita lógica extra dentro del componente para chequear si existe o no.



Para la coincidencia de rutas, React Router recorre la lista de rutas de arriba a abajo y renderiza la primera que coincide. Por eso el orden importa: puse /search/:initialSearchTerm antes de un posible catch-all, y al final tengo path="*" para el 404.

Parte 2: Hooks Avanzados y Estado: Explica los beneficios arquitectónicos de useContext, diseña la API para un hook personalizado useFetch y describe el propósito de la carga diferida.

useContext y ThemeContext

Antes, el tema vivía como estado en App.tsx y tenía que pasarlo por props hasta ThemeSwitcher. Eso está bien cuando la app es chica, pero si en algún momento necesito leer el tema desde un componente más adentro del árbol, tendría que pasarlo por varios niveles intermedios que no lo usan, lo que se llama prop drilling. Moví el estado a un contexto global para evitar eso.

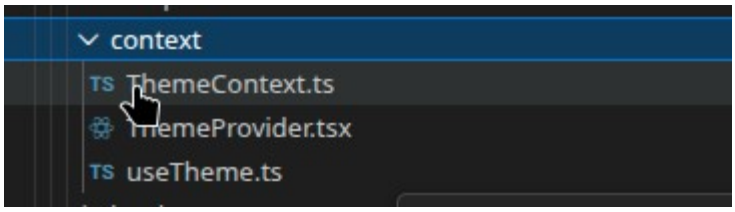
```
import { createContext } from 'react'

export type Theme = 'light' | 'dark'

export interface ThemeContextType {
  theme: Theme
  toggleTheme: () => void
}

export const ThemeContext = createContext<ThemeContextType | null>(null)
```

Lo dividí en tres archivos: ThemeContext.ts solo crea el contexto y define los tipos, ThemeProvider.tsx tiene el estado y el toggleTheme, y useTheme.ts es el hook que usan los componentes para consumirlo. Separarlo así hace que cada archivo tenga una responsabilidad clara



Usar useContext en lugar de pasar props resuelve un problema concreto: cuando el tema necesita llegar a componentes que están varios niveles adentro del árbol, tengo que pasarlo por componentes intermedios que no lo usan para nada. Con el contexto, cualquier componente lo consume directamente con useTheme() sin importar dónde esté en el árbol. La otra ventaja es que centralizar el estado en ThemeProvider hace que sea fácil de extender, si luego quiero que el tema persista en localStorage o leerlo desde las preferencias del sistema, solo modifico un archivo y el resto de la app no se entera.

useFetch

El hook recibe una URL y devuelve { data, isLoading, error }. La firma es genérica (useFetch<T>) para que funcione con cualquier tipo de respuesta. Adentro maneja tres estados: antes del fetch, durante, y después. También acepta null como URL, lo que permite "desactivarlo" sin condiciones raras afuera.

```
const { data, isLoading, error } = useFetch<OpenLibraryResponse>(url)
```

```typescript
import { useState, useEffect } from 'react'

interface FetchState<T> {
  data: T | null
  isLoading: boolean
  error: string | null
}

export function useFetch<T>(url: string | null): FetchState<T> {
  const [data, setData] = useState<T | null>(null)
  const [isLoading, setIsLoading] = useState<boolean>(false)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    if (!url) return

    let cancelled = false

    const fetchData = async () => {
      setIsLoading(true)
      setError(null)
      try {
        const res = await fetch(url)
        if (!res.ok) throw new Error(`HTTP error: ${res.status}`)
        const json: T = await res.json()
        if (!cancelled) setData(json)
      } catch (err) {
        if (!cancelled) setError(err instanceof Error ? err.message : 'Unknown error')
      } finally {
        if (!cancelled) setIsLoading(false)
      }
    }

    fetchData()
    return () => { cancelled = true }
  }, [url])

  return { data, isLoading, error }
}
```

Antes tenía esa lógica duplicada en SearchContainer y hubiera tenido que repetirla en BookDetailPage. Con el hook, los dos componentes simplemente llaman a useFetch y se desentienden de cómo funciona el fetch por dentro. Si en algún momento quiero agregar caché o retry, lo hago en un solo lugar.

```
EXPLORER
WEBDEV
capstone
  backend
  frontend / JunkanLibrary
    node_modules
    public
    src
      assets
      components
      context
        ThemeContext.ts
        ThemeProvider.tsx
        useTheme.ts
      hooks
        useFetch.ts
      pages
        AboutPage.scss
        AboutPage.tsx
        BookDetailPage.scss
        BookDetailPage.tsx
        HomePage.scss
        HomePage.tsx
        SearchPage.scss
        SearchPage.tsx
      types
      App.scss
      App.tsx
      index.scss
      main.tsx
    .gitignore
    eslint.config.js
```

SearchPage.tsx | README.md .../frontend/... | SearchPage.scss | AboutPage.tsx | AboutPage.scss | HomePage.tsx .../pages | HomeP

capstone > frontend > JunkanLibrary > src > pages > SearchPage.tsx > SearchPage

```
1  import { useState, useEffect } from 'react'
2  import { useParams, useNavigate } from 'react-router-dom'
3  import { useFetch } from '../hooks/useFetch'
4  import type { Book, OpenLibraryResponse } from '../types/book'
5  import BookList from '../components/BookList'
6  import './SearchPage.scss'
7
8  export default function SearchPage() {
9    const { initialSearchTerm } = useParams<{ initialSearchTerm?: string }>()
10   const navigate = useNavigate()
11
12   const [inputValue, setInputValue] = useState(initialSearchTerm ?? '')
13   const [query, setQuery] = useState(initialSearchTerm ?? 'kafka')
14
15   const url = query
16     ? `https://openlibrary.org/search.json?q=${encodeURIComponent(query)}&limit=20`
17     : null
18
19   const { data, isLoading, error } = useFetch<OpenLibraryResponse>(url)
20   const books: Book[] = data?.docs ?? []
21
22   useEffect(() => {
23     if (query) {
24       navigate(`/search/${encodeURIComponent(query)}`, { replace: true })
25     } else {
26       navigate('/search', { replace: true })
27     }
28   }, [query, navigate])
29
30   const handleSubmit = (e: React.FormEvent) => {
31     e.preventDefault()
32     if (inputValue.trim()) setQuery(inputValue.trim())
33   }
34
```

Lazy Loading

AboutPage y BookDetailPage los cargo con React.lazy(). Lo que esto hace es que Vite genera un bundle separado para cada uno, y el navegador solo lo descarga cuando el usuario navega a esa ruta. El resultado es que el bundle inicial de la app es más chico y carga más rápido. El <Suspense> envuelve las rutas y muestra un fallback mientras el chunk se descarga.

tone > frontend > JunkanLibrary > src > App.tsx > ...

```
import { lazy, Suspense } from 'react'
import { Routes, Route } from 'react-router-dom'
import { ThemeProvider } from './context/ThemeProvider'
import AppLayout from './components/AppLayout'
import HomePage from './pages/HomePage'
import SearchPage from './pages/SearchPage'

const AboutPage = lazy(() => import('./pages/AboutPage'))
const BookDetailPage = lazy(() => import('./pages/BookDetailPage'))

function LoadingFallback() {
  return (
    <div style={{ padding: '3rem', textAlign: 'center', color: 'var(--text-muted)' }}>
      Loading…
    </div>
  )
}

export default function App() {
  return (
    <ThemeProvider>
      <Suspense fallback={<LoadingFallback />}>
        <Routes>
          <Route element={<AppLayout />}>
            <Route path="/" element={<HomePage />} />
            <Route path="/search" element={<SearchPage />} />
            <Route path="/search/:initialSearchTerm" element={<SearchPage />} />
            <Route path="/books/:bookId" element={<BookDetailPage />} />
            <Route path="/about" element={<AboutPage />} />
            <Route path="*" element={
              <div style={{ textAlign: 'center', padding: '3rem', color: 'var(--text-muted)' }}>
                <h2>404 Page not found</h2>
              </div>
            } />
          </Route>
        </Routes>
      </Suspense>
    </ThemeProvider>
  )
}
```